

Sprawozdanie z projektu

Symulator Układu Automatemycznej Regulacji (UAR)

Sekcja 3 (3-osobowa) · Politechnika Śląska

1. Skład sekcji i podział obowiązków

Sekcja 3-osobowa. Podział obowiązków w realizacji projektu (część nieoceniana):

Członek sekcji	Zakres obowiązków
Konrad Konieczny	Warstwa komunikacji sieciowej — klasa Sieci: serwer/klient TCP (QTcpServer/QTcpSocket), projekt ramki protokołu i składanie ramek nad strumieniem TCP, serializacja binarna próbek (QDataStream) oraz konfiguracji (CBOR). Zarządzanie połączeniem i buforowaniem odbieranych danych.
Kacper Banasik	Logika symulacji rozproszonej w MainWindow — tryby taktowania jednostronny i obustronny, numeracja próbek (seq), wykrywanie gubienia i opóźnień pakietów, podtrzymanie ostatniej wartości (ZOH), lampka wydajności „wyrabia się / nie wyrabia się” oraz licznik pakietów na sekundę.
Jakub Siwy	GUI i integracja — blokowanie kontrolek w trybie sieciowym, zabezpieczenie przed przypadkową zmianą trybu pracy, automatyczne przesyłanie konfiguracji UAR, obsługa zerwania połączenia i powrotu do trybu stacjonarnego, testy na localhost, w sieci LAN i w połączeniu bezpośrednim.

Podział jest umowny — wszystkie krytyczne funkcjonalności sieciowe były projektowane, testowane i poprawiane wspólnie podczas spotkań online.

2. Protokół komunikacyjny, serializacja i wykrywanie gubienia pakietów

2.1. Architektura połączenia

Regulator pełni rolę serwera i nasłuchuje na porcie (domyślnie 45763), a instancja obiektu jest klientem, który się z nim łączy. Łączność działa na localhost, w sieci LAN z serwerem DHCP oraz w połączeniu bezpośrednim komputer–komputer. Do transmisji używamy gniazda TCP

(QTcpSocket, QTcpServer) z włączoną opcją małego opóźnienia oraz podtrzymania połączenia, co pozwala szybko wykryć zerwanie łącza.

2.2. Budowa ramki protokołu

Ponieważ TCP dostarcza ciągły strumień bajtów, a nie osobne pakiety, zastosowaliśmy własny sposób wydzielania wiadomości. Każda ramka składa się z nagłówka o stałym rozmiarze oraz payloadu o zmiennej długości. Nagłówek zawiera jeden bajt określający typ pakietu (konfiguracja albo próbka symulacji) oraz czterobajtową długość payloadu. Po stronie odbiorczej dane trafiają do bufora, a specjalna funkcja w pętli sprawdza, czy zgromadzono komplet (nagłówek i cały payload), i dopiero wtedy dekoduje pojedynczą ramkę. Dzięki temu poprawnie obsługujemy zarówno sklejone, jak i podzielone pakiety TCP.

2.3. Typy przesyłanych danych

O rodzaju usługi decyduje pole „rodzaj” w payloadzie. Wyróżniamy trzy typy wiadomości: sterowanie (regulator wysyła do obiektu sygnał sterujący u wraz z wartością zadaną, uchybem i składowymi PID), wyjście (obiekt odsyła regulatorowi wartość regulowaną y) oraz komendę (start, stop, reset sterujące przebiegiem symulacji po obu stronach). Każda próbka niesie też pola pomocnicze protokołu: numer kolejny (seq), czas symulacji, interwał oraz informacje diagnostyczne o synchronizacji.

2.4. Serializacja

Próbki symulacji są domyślnie serializowane binarnie za pomocą QDataStream, ze stałym układem pól i sygnaturą formatu — jest to sposób najszybszy i wymagany dla próbek. Konfiguracja UAR (parametry ARX, PID, generatora, interwał i tryb taktowania) jest kodowana binarnie w formacie CBOR. Alternatywnie użytkownik może przełączyć transmisję na format tekstowy JSON. Dekodowanie jest odporne — najpierw próbujemy odczytu binarnego, a w razie niepowodzenia traktujemy dane jako JSON. Wybór serializacji odbywa się w GUI, dzięki czemu można porównać wydajność obu formatów za pomocą wskaźnika liczby pakietów na sekundę.

2.5. Wykrywanie gubienia i opóźnień pakietów

TCP gwarantuje dostarczenie danych, więc przy zbyt małym interwale pakiety nie znikają, lecz docierają z opóźnieniem. Nasze mechanizmy wykrywają tę sytuację i pozwalają symulacji pracować dalej. Każda próbka ma numer kolejny (seq), na podstawie którego regulator wyznacza rozsynchronizowanie (różnicę między numerem wysłanym a odebrany) i porównuje czas, jaki upłynął, aby stwierdzić, czy któraś ze stron się opóźnia. W trybie jednostronnym, jeśli regulator wchodzi w kolejny takt zegara, a odpowiedź obiektu jeszcze nie dotarła, zapamiętuje ostatnią wartość regulowaną y i kontynuuje na niej pracę (podtrzymanie ZOH); na wykresie powstaje wtedy płaski odcinek, po którym od razu widać utratę lub opóźnienie pakietu. W trybie obustronnym obiekt, który nie otrzymał na czas nowego sterowania, trzyma poprzednią wartość u . Dodatkowo co sekundę zliczamy liczbę pakietów na sekundę i porównujemy ją z wartością oczekiwaną dla danego interwału — zielona lub czerwona „lampka” informuje operatora, czy

symulacja „się wyrabia”. Spóźnione i zdezaktualizowane próbki są odrzucane, aby wykres nie cofał się w czasie.

3. Historia rozwoju funkcjonalności sieciowych

Prace prowadziliśmy etapami wyznaczonymi przez kolejne spotkania projektowe:

1. **Spotkanie organizacyjne — omówienie regulaminu i podział pracy**
 - zapoznanie się z regulaminem projektu oraz wymaganiami tematu 1,
 - ustalenie zakresu funkcjonalności sieciowych do zrealizowania przez sekcję 3-osobową,
 - podział obowiązków między członków sekcji.
2. **I spotkanie seminaryjne — code-review, wstępny projekt protokołu i serializacja**
 - przejście aplikacji lokalnej (ARX, PID, generator, pętla UAR) od poprzedniej sekcji,
 - code-review oraz weryfikacja i walidacja check-listą, wykrycie i naprawa celowo wprowadzonych błędów,
 - opracowanie wstępnego projektu protokołu (nagłówek: typ i długość, payload z danymi),
 - napisanie klasy Sieci (serwer/klient TCP) i konsolowe przesłanie konfiguracji ARX/PID między dwoma stanowiskami.
3. **II spotkanie seminaryjne — korekta aplikacji i przesyłanie konfiguracji**
 - poprawa uchybień funkcjonalnych i jakościowych wykrytych na etapie walidacji,
 - wprowadzenie trybu sieciowego z blokowaniem kontrolek zgodnie ze specyfikacją,
 - widoczna sygnalizacja połączenia (wskaźnik stanu oraz informacja o adresie partnera),
 - zabezpieczenie zmiany trybu pracy przed przypadkowym kliknięciem (okno potwierdzenia),
 - automatyczne przesyłanie pełnej konfiguracji UAR przy każdej zmianie, z ochroną przed pętlą echo.
4. **III spotkanie seminaryjne — krytyczne funkcjonalności sieciowe**
 - uruchomienie symulacji UAR w sieci w trybie jednostronnym (jedna próbka wyjścia za jedną próbkę wejścia),
 - dodanie trybu obustronnego z niezależnym taktowaniem obu instancji, sterowanego komendami z regulatora,
 - umożliwienie wyboru serializacji binarnej i tekstowej oraz dodanie licznika pakietów na sekundę,
 - implementacja wykrywania opóźnień i gubienia pakietów (seq, rozsynchronizowanie, ZOH, lampka wydajności).
5. **IV spotkanie końcowe — walidacja aplikacji i zerowy termin odbiorów**

- usunięcie pozostałych usterek w działaniu aplikacji,
- testy poprawności na localhost, w sieci LAN oraz w połączeniu bezpośrednim,
- obsługa zerwania połączenia z automatycznym powrotem do trybu stacjonarnego,
- demonstracja działającej symulacji rozproszonej na komputerach w sali.

4. Najistotniejsze napotkane trudności i sposoby ich rozwiązania

- Składanie wiadomości nad TCP — protokół dostarcza strumień bajtów, więc pojedyncze ramki sklejały się lub były dzielone. Rozwiązaniem był własny nagłówek z typem i długością oraz buforowanie i składanie ramek po stronie odbiorczej.
- Gubienie i opóźnianie pakietów przy zbyt małym interwale — odpowiedzi nie docierały przed kolejnym taktem. Zastosowaliśmy podtrzymanie ostatniej wartości (ZOH) oraz wyraźną sygnalizację lampką, dzięki czemu symulacja pracuje dalej, a utratę widać na wykresie.
- Pętla echa przy synchronizacji konfiguracji — automatyczne odsyłanie ustawień powodowało nieskończoną wymianę komunikatów. Rozwiązała to flaga blokująca wysyłkę zwrotną w trakcie stosowania konfiguracji odebranej od partnera.
- Płynne przełączanie trybu sieciowy/stacjonarny — wymóg kontynuacji symulacji bez skoków. Osiągnęliśmy to, symulując nieużywane elementy UAR w tle i utrzymując identyczną konfigurację po obu stronach.

5. Czego nauczył się każdy członek sekcji

Konrad Konieczny

- Zasady działania TCP oraz projektowania własnego protokołu aplikacyjnego z wydzielaniem ramek nad strumieniem bajtów.
- Serializacja danych w Qt — binarna przez QDataStream oraz w formacie CBOR, wraz z odpornym dekodowaniem.
- Obsługa gniazd i modelu serwer/klient w Qt (QTcpServer, QTcpSocket, sygnały i sloty).

Kacper Banasik

- Synchronizacja symulacji rozproszonej oraz różnica między taktowaniem jednostronnym a obustronnym.
- Praktyczne radzenie sobie z opóźnieniami: podtrzymanie ostatniej wartości (ZOH) i śledzenie rozsynchronizowania na podstawie numerów próbek.
- Projektowanie prostych, czytelnych wskaźników jakości transmisji (lampka „wyrabia się”, pakiety na sekundę).

Jakub Siwy

- Projektowanie GUI odpornego na błędy operatora (blokowanie kontrolek, ochrona przed przypadkową zmianą trybu).
- Zarządzanie stanem aplikacji przy przełączaniu trybów sieciowy/stacjonarny bez utraty ciągłości symulacji.
- Testowanie aplikacji sieciowej w różnych środowiskach (localhost, LAN, połączenie bezpośrednie) oraz obsługa sytuacji awaryjnych, takich jak zerwanie połączenia.